

StudyBuddy

Ontwerpdocument

MBO niveau 4 - Software Development

Student: Yusif Atakishiyev

Project: StudyBuddy - studieportaal

Datum: 24 september 2026

Inhoudsopgave

1. Inleiding
2. Projectcontext en uitgangspunten
3. Functionele eisen
4. GUI-ontwerp
5. Use cases en flows
6. Technisch ontwerp
7. ERD
8. Flowdiagram
9. Ontwerpkeuzes en alternatieven
10. Security en privacy (AVG)
11. Testplan
12. Conclusie

1. Inleiding

Voor StudyBuddy maak ik een webapplicatie waarmee studenten hun vakken, cijfers, deadlines en studiemateriaal op één plek kunnen bekijken. Docenten of beheerders kunnen vakken, taken en materiaal beheren. In dit document leg ik uit hoe ik de applicatie wil bouwen voordat ik begin met programmeren.

Ik heb het ontwerp bewust simpel gehouden. De technieken sluiten aan bij wat ik tijdens mijn opleiding heb gebruikt: HTML, CSS, JavaScript, PHP en MySQL. Het ontwerp is zo geschreven dat de onderdelen later ook echt getest kunnen worden.

2. Projectcontext en uitgangspunten

Het belangrijkste doel is dat een student snel overzicht krijgt en niet op verschillende plekken hoeft te zoeken naar cijfers, deadlines en materiaal.

- De applicatie wordt een webapplicatie die op laptop en mobiel bruikbaar is.
- De backend wordt gemaakt in PHP en de database in MySQL.
- De frontend gebruikt HTML, eigen CSS, Tailwind CSS en een kleine hoeveelheid JavaScript. Voor een deel van de vormgeving heb ik zelf CSS geschreven. Op de login- en registratiepagina gebruik ik ook Tailwind CSS.
- Er zijn twee rollen: student en admin/docent.
- Alle databasebewerkingen lopen via PDO prepared statements.
- De applicatie slaat alleen gegevens op die nodig zijn voor het studieportaal.

3. Functionele eisen

Hieronder staan 12 functionele eisen. Bij elke eis staat wat het doel is, welke invoer nodig is, wat de uitvoer is en wat er gebeurt als iets fout gaat.

FE1 Inloggen

Doel: Een gebruiker veilig toegang geven tot StudyBuddy.

Invoer: Studentnummer/e-mail en wachtwoord.

Uitvoer: Bij juiste gegevens wordt de gebruiker doorgestuurd naar het juiste dashboard.

Foutafhandeling: Lege of verkeerde gegevens geven een duidelijke foutmelding. De gebruiker blijft op de loginpagina.

FE2 Student registreren

Doel: Een student een account laten aanmaken.

Invoer: Studentnummer, volledige naam, e-mail en wachtwoord.

Uitvoer: Account wordt opgeslagen en de student gaat naar de loginpagina.

Foutafhandeling: Dubbel studentnummer/e-mail of ongeldige invoer wordt niet opgeslagen en er verschijnt een melding.

FE3 Dashboard tonen

Doel: De student direct een overzicht geven.

Invoer: De ingelogde gebruiker; verder geen handmatige invoer.

Uitvoer: Vakken, gemiddeld cijfer, open taken en recente materialen worden getoond.

Foutafhandeling: Als er nog geen gegevens zijn, wordt een lege toestand met uitleg getoond in plaats van een fout.

FE4 Cijfer toevoegen

Doel: Een student een cijfer bij een vak laten vastleggen.

Invoer: Vak, cijfer van 1 t/m 10 en weging.

Uitvoer: Het cijfer verschijnt in de lijst en het gemiddelde wordt opnieuw berekend.

Foutafhandeling: Een cijfer buiten 1-10, ontbrekend vak of ongeldige weging wordt geweigerd.

FE5 Cijfers per vak bekijken

Doel: Cijfers overzichtelijk per vak kunnen bekijken.

Invoer: Gekozen vak of filter.

Uitvoer: Alle cijfers van dat vak en het berekende gemiddelde worden getoond.

Foutafhandeling: Bij geen cijfers verschijnt de tekst dat er nog geen cijfers zijn ingevoerd.

FE6 Taken bekijken en afronden

Doel: Studenten laten zien wat ze nog moeten doen.

Invoer: Taakstatus via checkbox of knop.

Uitvoer: Taken worden getoond met titel, beschrijving, deadline en status. Een afgeronde taak krijgt status voltooid.

Foutafhandeling: Een taak die niet bestaat of niet bij de student hoort kan niet worden aangepast.

FE7 Deadlinewaarschuwing tonen

Doel: Belangrijke deadlines sneller laten opvallen.

Invoer: Deadline uit de database en huidige datum/tijd.

Uitvoer: Open taken binnen 48 uur krijgen het label urgent.

Foutafhandeling: Een ontbrekende of ongeldige datum wordt niet als urgent gemarkeerd en kan door admin worden gecorrigeerd.

FE8 Studiemateriaal bekijken

Doel: Studenten materiaal per vak laten openen.

Invoer: Gekozen vak of materiaalitem.

Uitvoer: Titel, type en link naar het materiaal worden getoond.

Foutafhandeling: Een lege of ongeldige URL wordt niet als klikbare link weergegeven.

FE9 Student aan vak koppelen

Doel: Bepalen welke vakken een student ziet.

Invoer: Student en vak.

Uitvoer: De koppeling wordt opgeslagen in student_courses en het vak verschijnt bij de student.

Foutafhandeling: Een dubbele koppeling wordt tegengehouden.

FE10 Vak beheren door admin

Doel: Een admin vakken laten toevoegen, wijzigen en verwijderen.

Invoer: Vakcode, naam en beschrijving.

Uitvoer: De vakkenlijst wordt bijgewerkt na de gekozen actie.

Foutafhandeling: Verplichte velden mogen niet leeg zijn. Verwijderen vraagt eerst om bevestiging.

FE11 Taken en materiaal beheren door admin

Doel: Een admin inhoud per vak laten beheren.

Invoer: Taakgegevens of materiaalgegevens en het gekozen vak.

Uitvoer: Nieuwe of gewijzigde inhoud is zichtbaar op de detailpagina van het vak.

Foutafhandeling: Ongeldige deadline, lege titel of ongeldige URL wordt niet opgeslagen.

FE12 Rollen en toegangscontrole

Doel: Voorkomen dat studenten adminpagina's gebruiken.

Invoer: Ingelogde sessie en rol van de gebruiker.

Uitvoer: Student ziet studentpagina's; admin ziet ook beheerpagina's.

Foutafhandeling: Zonder geldige sessie volgt redirect naar login. Zonder adminrol volgt een melding of redirect naar dashboard.

4. GUI-ontwerp

De schermen krijgen dezelfde basisopbouw: bovenaan een eenvoudige header en aan de zijkant een menu op desktop. Op mobiel wordt het menu ingeklapt. Ik kies voor rustige schermen met duidelijke knoppen en niet te veel informatie tegelijk.

4.1 Login en registratie

In het midden staat een formulier. De gebruiker kan wisselen tussen inloggen en registreren. Foutmeldingen staan direct onder het veld waar iets fout is.

4.2 Dashboard

Bovenaan staan vier kleine overzichtskaarten: aantal vakken, gemiddeld cijfer, open taken en nieuw materiaal. Daaronder staan de vakken en de eerstvolgende deadlines.

4.3 Cijfers

Een tabel toont vak, cijfer, weging en datum. Boven de tabel staat een formulier om een cijfer toe te voegen en een filter op vak. Een voldoende en onvoldoende mogen visueel verschillen, maar er staat ook tekst bij zodat kleur niet de enige uitleg is.

4.4 Taken en deadlines

Taken staan in een lijst met titel, vak, deadline en status. Een open taak kan als voltooid worden gemarkeerd. Taken binnen 48 uur krijgen duidelijk het woord Urgent.

4.5 Materialen

Materiaal wordt per vak gegroepeerd. Elke kaart toont titel, bestandstype en een knop Openen.

4.6 Admin vakken beheren

De admin ziet een tabel met vakcode, vaknaam en acties. Toevoegen en wijzigen gebeurt via een duidelijk formulier. Verwijderen vraagt bevestiging.

4.7 Admin vakdetail

Op de vakdetailpagina staan twee onderdelen: taken en materialen. De admin kan daar nieuwe items toevoegen en bestaande items aanpassen.

Testbaarheid GUI: elk scherm is te controleren door de beschreven invoer uit te voeren en te kijken of de verwachte uitvoer of foutmelding zichtbaar wordt. Knoppen hebben een duidelijke naam, zoals Toevoegen, Opslaan, Verwijderen en Voltooid.

5. Use cases en flows

5.1 Use case: inloggen

1. Gebruiker opent de loginpagina.
2. Gebruiker vult studentnummer/e-mail en wachtwoord in.
3. De server controleert of de velden zijn ingevuld.
4. De server zoekt het account op met een prepared statement.
5. Het wachtwoord wordt gecontroleerd met `password_verify()`.
6. Bij succes wordt de sessie gestart en de rol gecontroleerd.
7. De gebruiker gaat naar het student- of admin dashboard.

Alternatief scenario: bij lege velden krijgt de gebruiker veldmeldingen. Bij verkeerde gegevens verschijnt een algemene melding dat de combinatie niet klopt. Er wordt niet verteld of alleen het account of alleen het wachtwoord fout is.

5.2 Use case: cijfer toevoegen

8. Student opent Mijn cijfers.
9. Student kiest een vak en vult cijfer en weging in.

10. PHP valideert de invoer.
11. Bij geldige invoer wordt het cijfer opgeslagen.
12. Het gemiddelde wordt opnieuw berekend en de tabel wordt vernieuwd.

Alternatief scenario: een cijfer buiten 1 t/m 10 of een lege weging wordt geweigerd. Er wordt niets in de database opgeslagen.

5.3 Use case: admin voegt vak toe

13. Admin opent Vakken beheren.
14. Admin kiest Nieuw vak.
15. Admin vult vakcode, naam en beschrijving in.
16. De server controleert de rol en invoer.
17. Het vak wordt opgeslagen en verschijnt in de tabel.

Alternatief scenario: bij een bestaande vakcode of lege naam verschijnt een foutmelding. Een student die de admin-URL handmatig invoert krijgt geen toegang.

6. Technisch ontwerp

De applicatie gebruikt een eenvoudige MVC-opbouw. Controllers verwerken acties, models praten met de database en views tonen HTML. Dit maakt het project overzichtelijker dan wanneer alle PHP, SQL en HTML in één bestand staan.

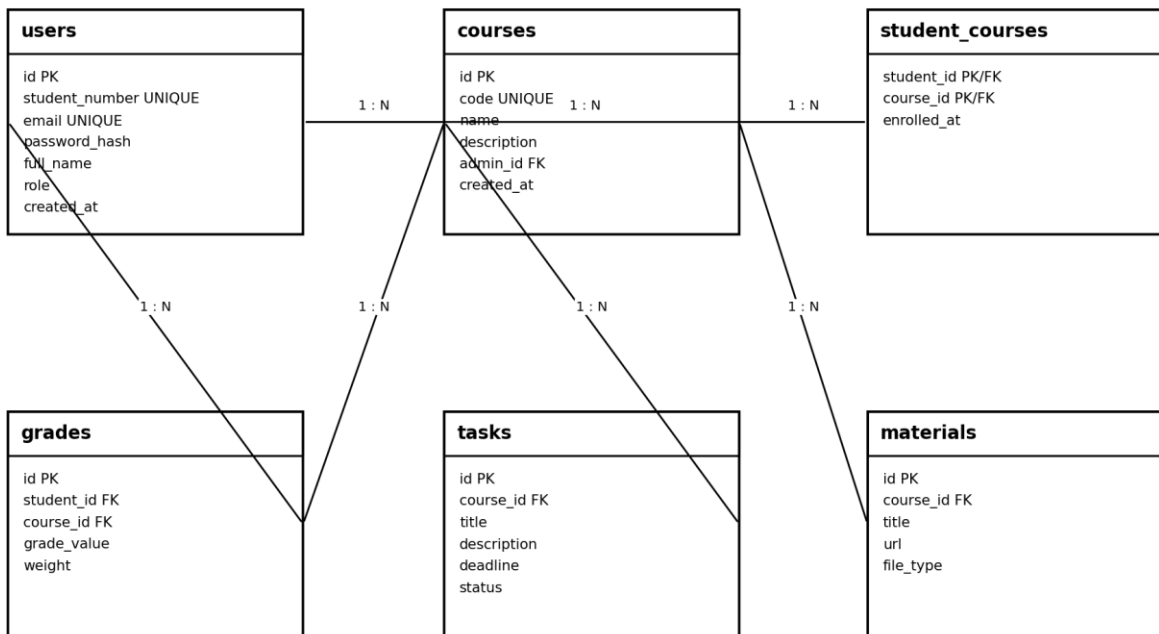
- Frontend: HTML5, eigen CSS, Tailwind CSS en JavaScript.
- Backend: PHP 8.x met objectgeoriënteerde classes.
- Database: MySQL.
- Databaseverbinding: PDO.
- Webserver tijdens ontwikkeling: Apache via een lokale LAMP/XAMPP-achtige omgeving.
- Sessies: PHP \$_SESSION met session_regenerate_id(true) na succesvol inloggen.

6.1 Bestandsstructuur

- config/Database.php - databaseverbinding
- src/controllers/AuthController.php, GradesController.php, CoursesController.php
- src/models/User.php, Course.php, Grade.php, Task.php, Material.php
- public/index.php en publieke assets
- views/ - login, dashboard, grades, tasks, materials en admin views
- public/css/style.css en public/js/app.js

7. ERD

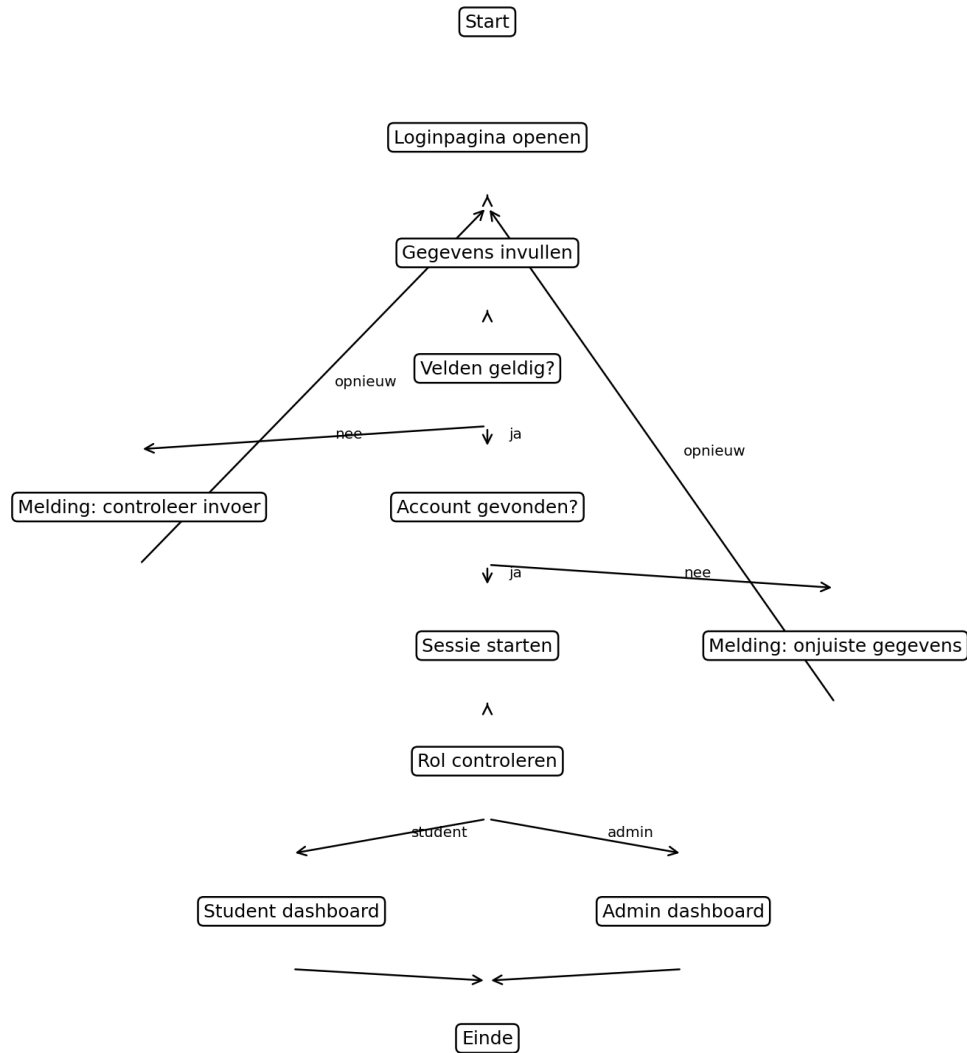
Het ERD hieronder is het databaseontwerp. PK betekent primary key en FK betekent foreign key. De koppeltabel student_courses lost de many-to-many relatie tussen studenten en vakken op.



Belangrijkste relaties: één gebruiker kan meerdere cijfers hebben; één vak kan meerdere cijfers, taken en materialen hebben; studenten en vakken hebben via student_courses een many-to-many relatie. courses.admin_id verwijst naar de gebruiker die het vak beheert.

8. Flowdiagram

Dit flowdiagram laat de volledige inlogroute zien, inclusief foutpaden en het verschil tussen student en admin.



9. Ontwerpkeuzes en alternatieven

Keuze 1 - Eigen CSS in combinatie met Tailwind CSS

Gekozen oplossing: Ik gebruik vooral mijn eigen CSS voor de kleuren, knoppen, formulieren, sidebar, kaarten en tabellen. Op de login- en registratiepagina gebruik ik daarnaast Tailwind CSS. Zo kan ik mijn eigen stijl maken, maar ook Tailwind gebruiken voor een snelle en nette opbouw van die pagina.

Alternatief: Bootstrap was ook een mogelijkheid. Daarmee kan ik kant-en-klare onderdelen gebruiken. Ik heb niet voor Bootstrap gekozen, omdat ik een groot deel van de styling zelf wilde maken. Voor de loginpagina gebruik ik Tailwind CSS als ondersteuning.

Keuze 2 - Eenvoudige PHP MVC-opbouw

Gekozen oplossing: Ik splits databasecode, logica en HTML op in models, controllers en views. Daardoor zijn bestanden kleiner en fouten makkelijker terug te vinden.

Alternatief: Laravel was een alternatief. Laravel heeft veel ingebouwde functies, maar is voor de omvang van dit schoolproject zwaarder en verbergt meer techniek die ik juist wil laten zien.

Keuze 3 Sessies voor inloggen

Gekozen oplossing: Na het inloggen bewaar ik de gebruikers-id en rol in een server-side PHP-sessie. Dit past bij een normale webapplicatie met PHP.

Alternatief: JWT-tokens waren een alternatief. Die zijn handig bij losse API-clients, maar StudyBuddy is één webapplicatie en heeft dat nu niet nodig.

Keuze 4 Koppeltabel voor student en vak

Gekozen oplossing: Ik gebruik student_courses omdat één student meerdere vakken kan volgen en één vak meerdere studenten kan hebben.

Alternatief: Een course_id direct in users zou eenvoudiger zijn, maar dan kan een student maar aan één vak gekoppeld worden. Dat past niet bij de eisen.

10. Security en privacy (AVG)

Omdat StudyBuddy persoonsgegevens en cijfers bevat, moet de applicatie zorgvuldig met gegevens omgaan. Ik neem daarom de volgende maatregelen:

- Wachtwoorden worden nooit als gewone tekst opgeslagen. Ik gebruik password_hash() met PASSWORD_ARGON2ID en controleer met password_verify().
- Alle SQL met gebruikersinvoer gebruikt PDO prepared statements om SQL-injectie te voorkomen.
- Tekst die uit de database in HTML wordt gezet, wordt waar nodig escaped met htmlspecialchars() tegen XSS.
- Adminroutes controleren altijd de sessie en de rol. Alleen een knop verbergen is niet genoeg.
- Na succesvol inloggen wordt het sessie-id vernieuwd om session fixation te beperken.
- Formulieren krijgen server-side validatie. JavaScript-validatie is alleen extra gebruiksgemak.
- De applicatie bewaart alleen naam, studentnummer, e-mail en studiegegevens die voor StudyBuddy nodig zijn. Geen telefoonnummer of adres.
- Een student mag alleen gegevens zien die bij zijn account en vakken horen.
- Bij verwijderen van een account moeten gekoppelde persoonsgegevens volgens de afgesproken bewaartermijn verwijderd of geanonimiseerd kunnen worden.

AVG-keuze: dataminimalisatie is het uitgangspunt. Ik verzamel dus niet meer gegevens dan nodig. Ook moet duidelijk zijn waarom de gegevens worden gebruikt en wie toegang heeft. Voor een echte schoolomgeving moet daarnaast een bewaartermijn en procedure voor inzage/verwijdering worden afgesproken met de school.

11. Testplan

Test	Eis	Actie/invoer	Verwacht resultaat
T1	FE1	Correct account	Dashboard wordt geopend
T2	FE1	Verkeerd wachtwoord	Foutmelding, geen sessie
T3	FE2	Bestaand e-mailadres	Registratie wordt geweigerd
T4	FE4	Cijfer 7,5 met geldige	Cijfer opgeslagen en

		weging	gemiddelde aangepast
T5	FE4	Cijfer 12	Foutmelding, niets opgeslagen
T6	FE6	Open taak als voltooid markeren	Status wordt voltooid
T7	FE7	Open deadline binnen 48 uur	Label Urgent zichtbaar
T8	FE9	Zelfde student opnieuw aan hetzelfde vak koppelen	Dubbele koppeling geweigerd
T9	FE10	Admin maakt geldig vak	Vak verschijnt in tabel
T10	FE10	Student opent admin-URL	Toegang geweigerd/redirect
T11	FE8	Materiaal met geldige URL	Openen-knop werkt
T12	FE12	Niet ingelogde gebruiker opent dashboard	Redirect naar login

12. Conclusie

Met dit ontwerp heb ik vooraf duidelijk gemaakt wat StudyBuddy moet kunnen en hoe ik het technisch wil opbouwen. De 12 functionele eisen zijn testbaar uitgewerkt. Ook zijn de schermen, belangrijkste gebruikersflows, database, beveiliging en ontwerpkeuzes beschreven. Tijdens het bouwen kan ik dit document gebruiken als controle om te kijken of de applicatie nog volgens het ontwerp wordt gemaakt.